

Modul 3 – Operasi Citra Sederhana

Pengolahan Citra dan Visi Komputer

Dosen Pengampu Mata Kuliah

Prof. Cahya Rahmad, S.T., M.Kom., Dr. Eng.



Faiva Puspa Sahara

244107020036

TI – 3A

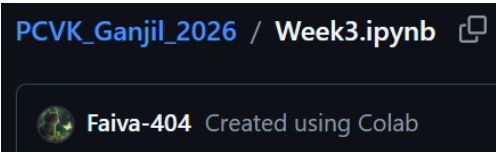
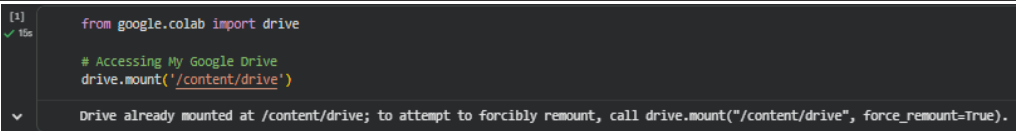
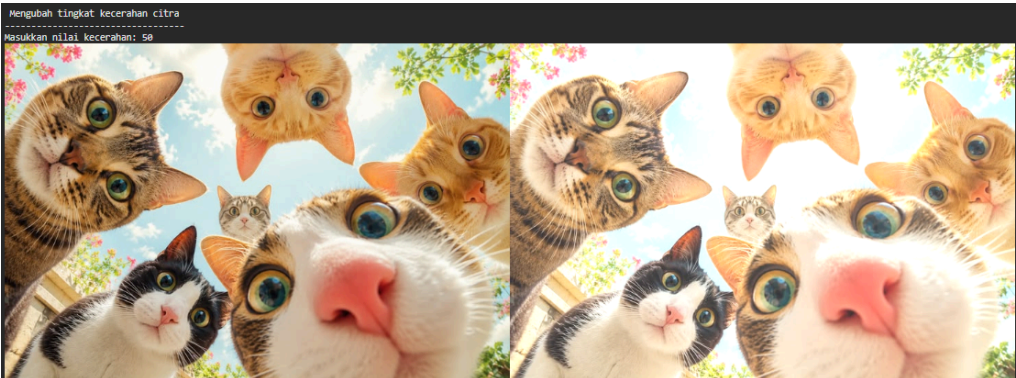
PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

D1. Operasi Citra Sederhana

Langkah	Jawaban/Deskripsi
1	
2	
3	

Tugas Praktikum

No.	Jawaban
1	Implementasikan inverse citra pada Google Colaboratory menggunakan formula yang terdapat pada bagian Ulasan Teor 
2	Implementasikan transformasi contrast pada Google Colaboratory menggunakan formula yang terdapat pada bagian Ulasan Teori untuk kontras

	<pre> brightness = int(input('Masukkan tingkat kecerahan: ')) contrast = float(input('Masukkan kontras: ')) original = cv.imread('/content/drive/MyDrive/SEMESTER 5/Citra/Kucing.jpg') # Hitung Correction Factor (F) bertipe float F = (259 * (contrast + 255)) / (255 * (259 - contrast)) # Konversi original ke float agar aman dari overflow contrast_img = F * (original.astype(float) - 128) + 128 + brightness contrast_img = np.clip(contrast_img, 0, 255).astype(np.uint8) final_frame = cv.hconcat((original, contrast_img)) cv2.imshow('final_frame') Masukkan tingkat kecerahan: 50 Masukkan kontras: 50 </pre> 
3	Implementasikan transformasi logarithmic brightness pada Google Colaboratory menggunakan formula yang terdapat pada bagian Ulasan Teori untuk transformasi log <pre> c = float(input('Masukkan nilai kecerahan: ')) original = cv.imread('/content/drive/MyDrive/SEMESTER 5/Citra/Hamster.jpg') # Hitung logaritma dengan casting ke float64[cite: 1] log_img = c * np.log(1.0 + original.astype(np.float64)) log_img = np.clip(log_img, 0, 255).astype(np.uint8) final_frame = cv.hconcat((original, log_img)) cv2.imshow('final_frame') Masukkan nilai kecerahan: 50 </pre> 
4	Implementasikan transformasi grayscale menggunakan metode averaging, lightness, dan luminance pada Google Colaboratory menggunakan formula


```

original = cv.imread('/content/drive/MyDrive/SEMESTER 5/Citra/kucing.jpg')

# Pisahkan Channel B, G, R (OpenCV menggunakan format BGR)
B, G, R = original[:, :, 0].astype(float), original[:, :, 1].astype(float), original[:, :, 2].astype(float)

# a. Averaging[cite: 1]
gray_avg = ((R + G + B) / 3).astype(np.uint8)

# b. Lightness[cite: 1]
gray_light = ((np.maximum.reduce([R, G, B]) + np.minimum.reduce([R, G, B])) / 2).astype(np.uint8)

# c. Luminance[cite: 1]
gray_lum = (0.21 * R + 0.72 * G + 0.07 * B).astype(np.uint8)

# Tampilkan salah satu contoh (Luminance)
cv2.imshow(cv.hconcat((original, cv.cvtColor(gray_avg, cv.COLOR_GRAY2BGR))))
cv2.imshow(cv.hconcat((original, cv.cvtColor(gray_light, cv.COLOR_GRAY2BGR))))
cv2.imshow(cv.hconcat((original, cv.cvtColor(gray_lum, cv.COLOR_GRAY2BGR))))

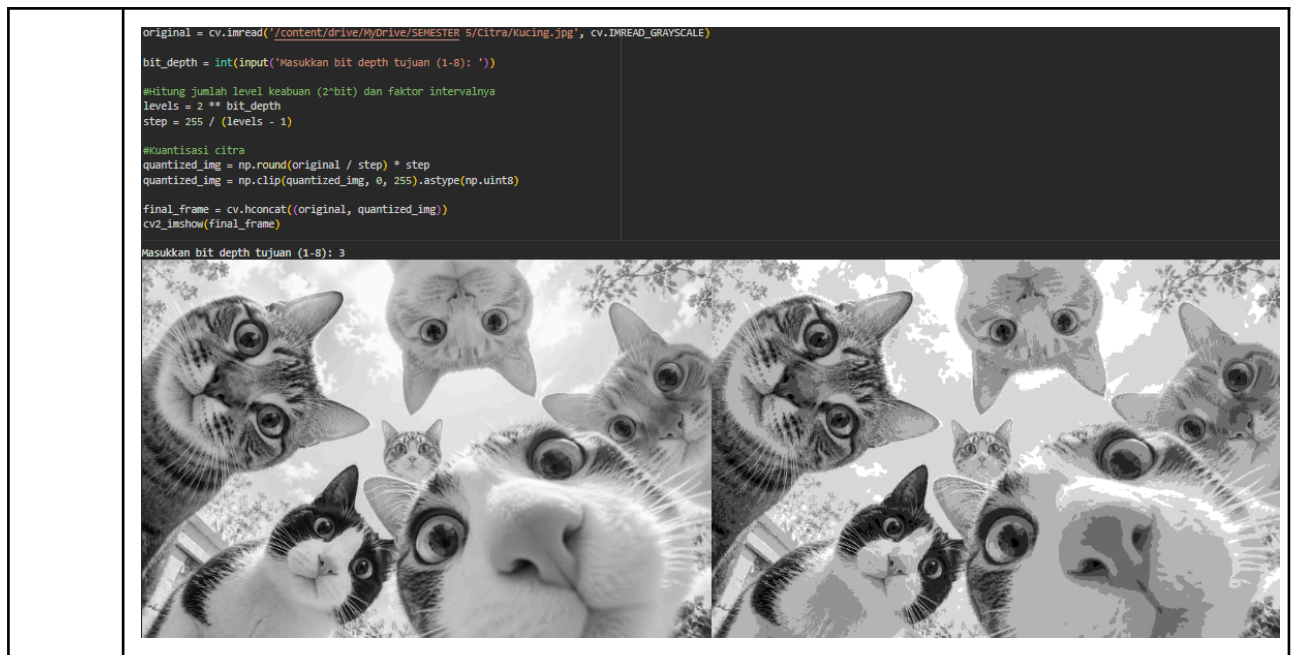
```



5

Tampilkanlah warna tertentu pada citra, dan ubah warna lain menjadi grayscale.

	<pre> original = cv.imread('/content/drive/MyDrive/SEMESTER 5/Citra/kucing.jpg') hsv = cv.cvtColor(original, cv.COLOR_BGR2HSV) #Definisikan rentang warna PINK / Merah Muda dalam HSV lower_pink = np.array([0, 50, 100]) upper_pink = np.array([15, 255, 255]) #Buat mask warna pink/merah hidung mask_pink = cv.inRange(hsv, lower_pink, upper_pink) #Buat citra grayscale versi 3 channel (BGR) gray = cv.cvtColor(original, cv.COLOR_BGR2GRAY) gray_3ch = cv.cvtColor(gray, cv.COLOR_GRAY2BGR) #Filter: Jika piksel bernilai pink ambil warna asli, sisanya jadi grayscale result = np.where(mask_pink[:, :, None] == 255, original, gray_3ch) #Tampilkan perbandingan final_frame = cv.hconcat((original, result)) cv2_imshow(final_frame) </pre> 
6	Buat Gamma Correction <pre> original = cv.imread('/content/drive/MyDrive/SEMESTER 5/Citra/Hamster.jpg') try: #Input nilai gamma (contoh: 0.5 untuk menerangkan, 2.0 untuk menggelapkan) gamma = float(input('Masukkan nilai Gamma (misal 0.5 atau 2.0): ')) #Normalisasi nilai piksel ke range 0.0 - 1.0, hitung pangkat gamma, lalu kembalikan ke 0 - 255 normal_img = original / 255.0 gamma_img = np.power(normal_img, gamma) * 255.0 gamma_img = np.clip(gamma_img, 0, 255).astype(np.uint8) #Tampilkan perbandingan (Asli vs Gamma Correction) final_frame = cv.hconcat((original, gamma_img)) cv2_imshow(final_frame) except ValueError: print('Error: Input harus berupa angka/float!') </pre> Masukkan nilai Gamma (misal 0.5 atau 2.0): 0.5 
7	Buat Simulasi Image Depth



8. Buat modul Average Denoising

```

original = cv.imread('/content/drive/MyDrive/SEMESTER 5/Citra/Kucing.jpg')
h, w = original.shape[:2]

scale_percent = float(input('Masukkan skala reduksi % (misal 10 untuk 10%): '))

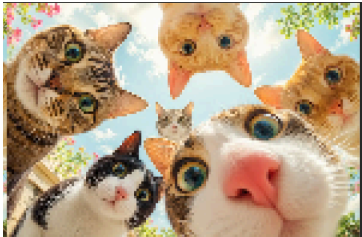
# Resize menjadi lebih kecil
new_w = int(w * scale_percent / 100)
new_h = int(h * scale_percent / 100)
small_img = cv.resize(original, (new_w, new_h), interpolation=cv.INTER_LINEAR)

# Resize balik ke ukuran asli untuk melihat efek penurunan resolusi spasial
spatial_res_img = cv.resize(small_img, (w, h), interpolation=cv.INTER_NEAREST)

final_frame = cv.hconcat((original, spatial_res_img))
cv2_imshow(final_frame)

```

Catat hasil PSNR pada tabel berikut.

No.	Jumlah Citra di Avg	Image Hasil	Nilai PNSR (dB)
1.	10		19.56 dB
2.	20		22.36 dB

3.	40		26.60 dB
4.	80		29.07 dB
5.	100		inf dB

- 8.1. Apakah peningkatan jumlah citra selalu memberikan peningkatan PSNR yang signifikan?
- Tidak selalu. Peningkatan PSNR sangat signifikan pada penambahan jumlah citra awal (misalnya dari 10 ke 20 citra). Namun, saat jumlah citra sudah semakin banyak (misalnya dari 80 ke 100 citra), peningkatannya mulai melambat (*diminishing returns*).
- 8.2. Pada jumlah citra berapa peningkatan mulai tidak terlalu signifikan?
- Peningkatan mulai tidak terlalu signifikan saat jumlah citra mencapai 40 hingga 80 citra. Pada titik ini, *noise* sudah tereduksi secara dominan sehingga penambahan citra ekstra hanya memberikan kenaikan nilai PSNR yang sangat kecil.
- 8.3. Apa Kesimpulan yang diperoleh dari percobaan ini?
- Metode *Average Denoising* sangat efektif untuk menghilangkan *Gaussian Noise* karena rata-rata dari nilai *noise* acak berkonsentrasi mendekati angka nol.
 - Semakin banyak citra bernoise yang dirata-ratakan, semakin tinggi nilai PSNR dan semakin jernih citra yang dihasilkan.
 - Terdapat batas efisiensi secara matematis; menambahkan jumlah citra secara terus-menerus tidak lagi memberikan peningkatan kualitas citra secara drastis, tetapi membutuhkan daya komputasi yang lebih besar.

9. Buat image masking

```
img = cv.imread("../content/drive/MyDrive/SEMESTER 5/Citra/Rapunzel.jpg")
h, w, c = img.shape

#Buat mask biner dengan 2 lingkaran di area wajah
mask = np.zeros((h, w), dtype=np.uint8)

#Lingkaran 1: Wajah Rapunzel (sebelah kiri)
cv.circle(mask, (int(w * 0.42), int(h * 0.48)), int(w * 0.12), 255, -1)

#Lingkaran 2: Wajah Flynn Rider (sebelah kanan)
cv.circle(mask, (int(w * 0.60), int(h * 0.23)), int(w * 0.12), 255, -1)

#Lakukan masking menggunakan operator bitwise AND
masked_image = cv.bitwise_and(img, img, mask=mask)

#Konversi mask ke 3 channel untuk tampilan hconcat
mask_3ch = cv.cvtColor(mask, cv.COLOR_GRAY2BGR)

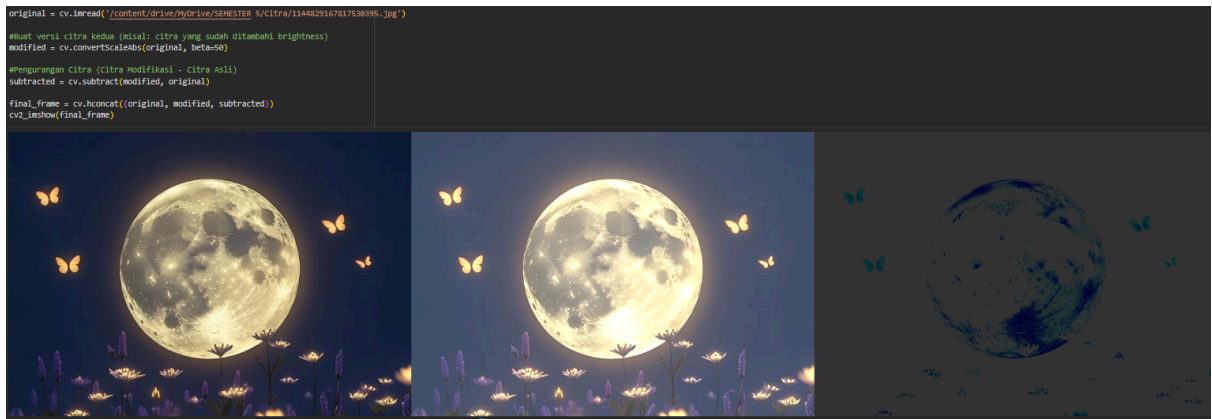
#Tampilkan Citra Asli, Mask, dan Hasil Masking
final_frame = cv.hconcat((img, mask_3ch, masked_image))
cv2.imshow('final_frame')
```



Lakukan percobaan menggunakan operator lain dan tunjukkan hasilnya

No.	Operator	Image Input	Image Output
1.	NOT (komplemen)		
2.	OR (atau)		
3.	AND (dan)		
4.	N AND (Not And)		
5.	XOR (Exlusive Or)		

10. Foto Malam Hari



10.1. Metode yang Dipilih = Linear Brightness

10.2. Alasan menggunakan linear brightness

→ Meningkatkan Intensitas Secara Merata, menambahkan konstanta pada setiap piksel secara linier untuk menerangkan seluruh bagian foto yang gelap agar detail seperti bulan dan bunga di latar bawah terlihat lebih jelas.

10.3. Resiko metode

→ *Overexposure/Blown-out Highlights*, area yang dari awal sudah terang (seperti permukaan bulan) akan ikut bertambah terang hingga mencapai nilai maksimum #255 (putih polos), sehingga detail tekstur kawah di dalam bulan justru menjadi hilang/terpotong (*clipping*).

→ *Efek Flatted/Washed Out*, karena area hitam pudar menjadi abu-abu gelap, gambar kehilangan kontras alaminya.

11. Buka gambar crayfish.jpg pada folder Images.



11.1. Metode yang dipilih

→ Kombinasi Gamma Correction dan Contrast Adjustment

11.2. Alasan pemilihan

→ Citra *crayfish* berada di lingkungan bawah air (*underwater*) yang cenderung gelap dan memiliki kontras rendah. Gamma correction digunakan untuk mengangkat detail tersembunyi pada area *shadow* tanpa membuat warna lobster menjadi pudar (*overexposed*), sedangkan pengaturan kontras berfungsi memperjelas tekstur cangkang dan warna merah lobster dari latar belakang laut.

11.3. Menentukan parameter terbaik

- Parameter $\gamma = 0.7$ dipilih karena memberikan kecerahan optimal pada area latar yang gelap.
- Parameter $\alpha = 1.2$ dipilih untuk menaikkan kontras secara pas agar warna merah lobster terlihat lebih pekat dan tajam tanpa menimbulkan *noise* berlebih.

11.4. Menghasilkan citra dengan menunjukkan before-after

